

Design Pattern Report: Observer Pattern

Group 52

Nguyễn Đình Minh Huy (Student ID: 25125083)

July 15, 2026

Contents

1	Real-world Problem	2
2	Naive Solution (Without Design Pattern)	2
3	Problems with the Naive Solution	3
4	Introduction to the Observer Pattern	3
5	General Class Diagram	4
6	Class Diagram for the Bank Account Problem	4
7	Class Diagram for the Bank Account Problem	4
8	Implementation of the Observer Pattern	4
9	Pros and Cons	6
9.1	Pros	6
9.2	Cons	6
10	Other Real-world Applications	6
10.1	Web Development	6
10.2	Mobile Development	6
11	Quiz	6
12	References	7

1 Real-world Problem

Consider a modern banking application where users perform transaction operations such as deposits on their bank accounts. Whenever a transaction occurs (e.g., a deposit is made), the system needs to notify various downstream services to process this event:

- Update the user interface (UI) to reflect the new balance.
- Send an email notification/receipt to the user.
- Log the transaction details for auditing and record-keeping.
- Run real-time fraud detection analytics to ensure transaction safety.
- Send a mobile push notification to the user's phone.
- Forward data to analytics systems for user behavior reporting.

The core challenge is establishing a system where a bank account can broadcast transaction updates to a dynamic set of interested services/observers without tightly coupling the core banking logic to concrete service classes or notification mechanisms.

2 Naive Solution (Without Design Pattern)

In a naive implementation, the `NaiveBankAccount` class directly references and invokes methods on concrete service classes whenever a deposit is made. Below is the C++ representation of this approach:

```
1 // Naive implementation demonstrating tight coupling
2 class NaiveBankAccount {
3 private:
4     double balance;
5     std::string accountNumber;
6
7     // Direct dependencies on concrete notification/processing services
8     UIUpdateService* uiService;
9     EmailService* emailService;
10    LoggerService* loggerService;
11    FraudDetectionService* fraudService;
12    MobileAppService* mobileService;
13    AnalyticsService* analyticsService;
14
15 public:
16     NaiveBankAccount(std::string accNum)
17         : balance(0.0), accountNumber(accNum),
18           uiService(new UIUpdateService()),
19           emailService(new EmailService()),
20           loggerService(new LoggerService()),
21           fraudService(new FraudDetectionService()),
22           mobileService(new MobileAppService()),
23           analyticsService(new AnalyticsService()) {}
24
25     ~NaiveBankAccount() {
26         delete uiService; delete emailService; delete loggerService;
27         delete fraudService; delete mobileService; delete
analyticsService;
```

```

28     }
29
30     void deposit(double amount) {
31         balance += amount;
32
33         // Direct method invocation on concrete objects
34         uiService->updateUI(amount, balance);
35         emailService->sendEmail(accountNumber, amount);
36         loggerService->logTransaction(accountNumber, amount);
37         fraudService->detectFraud(accountNumber, amount);
38         mobileService->pushNotification(amount);
39         analyticsService->trackEvent("deposit", amount);
40     }
41 };

```

Listing 1: Naive Implementation of Bank Account

3 Problems with the Naive Solution

The naive approach exhibits several major architectural flaws:

1. **Tight Coupling:** The `NaiveBankAccount` is tightly coupled to concrete classes like `UIUpdateService`, `EmailService`, etc. It must hold references to each concrete service and know exactly which methods to invoke on them.
2. **Violation of the Open-Closed Principle (OCP):** If we want to introduce a new downstream service (such as a regulatory auditing service or a tax calculator), we must modify the `NaiveBankAccount` class. This requires adding a new pointer, allocating it in the constructor, deleting it in the destructor, and invoking it inside `deposit`.
3. **Violation of the Single Responsibility Principle (SRP):** The bank account class should focus solely on managing balance and basic transaction logic. Instead, it is tasked with orchestrating notification emails, logging, UI rendering, and fraud checks.
4. **Rigidity:** Since the dependencies are hardcoded, they cannot be configured or swapped at runtime, making unit testing difficult (cannot mock dependencies).

4 Introduction to the Observer Pattern

The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects. When one object (the **Subject** or **Publisher**) changes state, all its dependents (the **Observers** or **Subscribers**) are notified and updated automatically.

Key concepts:

- **Subject:** Knows its observers and provides an interface to attach or detach them.
- **Observer:** Defines an updating interface for objects that should be notified.
- **ConcreteSubject:** Stores state of interest to `ConcreteObservers` and sends notifications.

- **ConcreteObserver:** Implements the Observer updating interface to keep its state consistent with the subject.

5 General Class Diagram

In the general Observer pattern structure:

- A **Subject** maintains a list of references to **Observer** objects.
- The **Subject** exposes interfaces: `attach(Observer)`, `detach(Observer)`, and `notify()`.
- **Observer** exposes the `update()` method interface.



Figure 1: General Observer Design Pattern UML Diagram

6 Class Diagram for the Bank Account Problem

7 Class Diagram for the Bank Account Problem

Applying the pattern to our problem:

- **BankAccount** acts as the Subject.
- **TransactionObserver** acts as the Observer interface.
- **UIObserver**, **EmailObserver**, **LoggerObserver**, **FraudAIObserver**, and **AnalyticsObserver** act as the ConcreteObservers.



Figure 2: UML Class Diagram for Bank Account Notification System

8 Implementation of the Observer Pattern

By decoupling the classes through abstract interfaces, our refactored implementation allows adding new service observers at runtime without modifying the subject class. We also utilize modern C++ smart pointers (`std::shared_ptr`) to safely manage object lifetimes.

```

1 // Observer interface
2 class TransactionObserver {
3 public:
4     virtual ~TransactionObserver() = default;
5     virtual void update(double amount, double balance) = 0;
6 };
7
8 // Concrete Observers
9 class UIObserver : public TransactionObserver {
10 public:
11     void update(double amount, double balance) override {
12         std::cout << "[UI] Updated. Transaction: " << amount
13             << ", New Balance: " << balance << "\n";
14     }
15 };
16
17 class EmailObserver : public TransactionObserver {
18 public:
19     void update(double amount, double balance) override {
20         std::cout << "[Email] Sent receipt. Amount: " << amount
21             << ", Current Balance: " << balance << "\n";
22     }
23 };
24
25 // Subject / Publisher
26 class BankAccount {
27 private:
28     double balance = 0.0;
29     std::string accountNumber;
30     std::vector<std::shared_ptr<TransactionObserver>> observers;
31
32 public:
33     BankAccount(std::string accNum) : accountNumber(accNum) {}
34
35     void attach(std::shared_ptr<TransactionObserver> observer) {
36         observers.push_back(observer);
37     }
38
39     void detach(std::shared_ptr<TransactionObserver> observer) {
40         observers.erase(
41             std::remove(observers.begin(), observers.end(), observer),
42             observers.end()
43         );
44     }
45
46     void deposit(double amount) {
47         balance += amount;
48         notify(amount);
49     }
50
51     void notify(double amount) {
52         for (const auto& observer : observers) {
53             observer->update(amount, balance);
54         }
55     }
56 };

```

Listing 2: Refactored Observer Implementation

9 Pros and Cons

9.1 Pros

- **Loose Coupling:** The Subject and Observers are loosely coupled. The Subject only knows that the observer implements a specific interface.
- **Support for Broadcast Communication:** Unlike direct calls, notifications are broadcast automatically to all registered observers.
- **Open-Closed Principle:** We can introduce new concrete observer classes without changing the subject's code.

9.2 Cons

- **Lapsed Listener Problem:** Observers must be carefully deregistered when destroyed, or they remain in memory and get called, causing undefined behavior or leaks.
- **Unintended Update Chains:** If observers trigger updates on subjects, it can lead to complex and hard-to-debug cascading updates or infinite loops.
- **Ordering Concerns:** There is no guarantee in which order the observers will be notified.

10 Other Real-world Applications

10.1 Web Development

- **Event Handling in JavaScript:** The DOM's `addEventListener` uses the Observer pattern to attach event handlers to HTML elements.
- **State Management (Redux/Vuex):** Views subscribe to changes in a global store and re-render automatically when state mutations occur.

10.2 Mobile Development

- **RxSwift / RxJava / RxKotlin:** Reactive streams rely heavily on observing data sequences and binding them directly to UI components.
- **SwiftUI/Android Jetpack Compose:** Declarative UI frameworks use property wrappers like `@Published` and state flows to push model updates directly to the view.

11 Quiz

Here are some interactive questions to review the Observer pattern:

1. What type of design pattern is the Observer Pattern?

- A) Creational
 - B) Structural
 - C) Behavioral
 - D) Architectural
2. Which SOLID principle does the Observer pattern primarily help satisfy when adding new observers?
- A) Liskov Substitution Principle
 - B) Single Responsibility Principle
 - C) Interface Segregation Principle
 - D) Open-Closed Principle
3. What is the “Lapsed Listener” problem in the Observer pattern?
- A) The subject stops listening to updates.
 - B) Observers are not notified because the subject crashed.
 - C) An observer is deleted without unsubscribing, causing memory leaks/crashes.
 - D) Multiple subjects occupy the same listener list.
4. Which of the following is a key advantage of the Observer pattern?
- A) It ensures observers are updated sequentially.
 - B) It decouples the subject from concrete observer details.
 - C) It reduces memory overhead of notifications.
 - D) It prevents subjects from changing their state.

Answer Key: 1-C, 2-D, 3-C, 4-B.

12 References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Observer*. <https://refactoring.guru/design-patterns/observer>